

Wait Less

April 10 Slide Assets Pack

Sapienza IoT Project Team

April 9, 2026

This PDF is a presentation-support handout. It collects the exact tables, numbers, and charts that can be copied directly into the April 10 slides.

1. Checkpoint Message

Main message for the presentation: We already validated the adaptive controller in software and validated node-to-node LoRa communication on real hardware. The remaining work is mainly sensor calibration, actuator integration, and full end-to-end validation.

2. Optional Technical Slide: Code Structure And Implementation Approach

Module or file group	Responsibility in the system
<code>firmware/node_a/main.cpp</code>	Reads side A sensors, builds telemetry, supports emulation and debug commands, and transmits compact LoRa packets.
<code>firmware/node_b/main.cpp</code>	Reads side B sensors, receives Node A telemetry, runs the controller, drives the traffic-light LEDs, and reports hardware status.
<code>lib/traffic_control/ TrafficTypes.h</code>	Defines the shared data model: telemetry, light outputs, and controller decisions.
<code>lib/traffic_control/ AdaptiveController.*</code>	Contains the phase logic: minimum green, maximum green, yellow transition, demand comparison, and emergency priority handling.
<code>lib/traffic_control/ SensorSupport.*</code>	Isolates low-level ultrasonic reading and threshold-based occupancy detection.
<code>lib/traffic_control/ NodeMessaging.*</code>	Encodes and decodes the compact telemetry format exchanged between the two nodes.
<code>lib/traffic_control/ DebugSupport.h</code>	Provides structured log modes and report commands so testing stays readable and repeatable.

Table 1: Code structure ready to use in a slide about implementation organization.

Short script to say on this slide

We wrote the code in layers so that each responsibility is isolated. Sensor reading is separated from queue estimation, messaging is separated from control logic, and the shared controller sits in reusable library files. This made it possible to test the policy first in software, then reuse the same logic in the ESP32 firmware, and finally validate the communication path on real boards.

Step	Implementation approach used in the project
1	Read raw ultrasonic distances on each node and convert them into far/near occupancy states using explicit thresholds (far sensor placed ~ 40 m upstream; each sensor uses a 100 cm detection window around itself).
2	Estimate queue state from sensor events rather than switching directly on a single binary trigger.
3	Keep the embedded logic shared in reusable library files so the simulator and firmware follow the same policy.
4	Make Node A a thin sensing and telemetry node, and keep Node B as the control and actuation node.
5	Exchange a compact payload over LoRa instead of sending large or cloud-dependent messages.
6	Decide switching locally on Node B using measurable timing rules: minimum green, yellow, and maximum green.
7	Add structured debug and report modes so each experiment can be repeated and documented clearly.

Table 2: Implementation approach to explain how the code was written and why it is modular.

3. Slide 2: Baseline Requirements

ID	Requirement	Current value
R1	Each side is represented by two ultrasonic detections: one far and one near.	2 detections per side
R2	The controller evaluates traffic demand periodically.	200 ms
R3	Node A produces periodic telemetry for Node B.	1000 ms
R4	The controller does not switch before a minimum green interval.	5000 ms
R5	Every switch includes a yellow safety interval.	2000 ms
R6	Starvation is limited by a maximum green cap if the other side remains waiting.	20000 ms
R7	Switching is demand-based rather than purely fixed-cycle.	Score = 3 * queue + 2 * farOccupied + 4 * nearOccupied

Table 3: Checkpoint requirements derived from the current implementation baseline.

Baseline parameter	Current value	Use in the talk
Control loop interval	200 ms	State this as the current controller reaction baseline.
Telemetry period	1000 ms	State this as the remote-data freshness baseline.
Far threshold	100 cm	Detection window around the far sensor (placed ~ 40 m upstream).
Near threshold	100 cm	Detection window around the near sensor (at the stop line).
Minimum green	5000 ms	State this as the anti-oscillation rule.
Yellow interval	2000 ms	State this as the safety transition rule.
Maximum green	20000 ms	State this as the anti-starvation rule.
Switching margin	4 points	State this as the current comparison threshold between sides.
Telemetry example	A,1,0,4,2,2,0,12345	Use this to show the packet format.
Payload size	19 bytes	Use this to show communication compactness.

Table 4: Exact baseline numbers to place in the requirement slide.

Sensor zone	Zone length	Max speed implied by 200 ms	Assumed max speed	Required loop for that speed
Far threshold (100 cm)	1.00 m	≤ 5.0 m/s ≈ 18 km/h	30 km/h (8.33 m/s)	$\leq 1.00/8.33$ s ≈ 120 ms
Near threshold (100 cm)	1.00 m	≤ 5.0 m/s ≈ 18 km/h	15 km/h (4.17 m/s)	$\leq 1.00/4.17$ s ≈ 240 ms

Table 5: Sampling-interval justification: current 200 ms loop assumes near-stop traffic; higher approach speeds require a faster loop or a wider detection zone.

4. Slide 3: Metrics Table

Metric	Current value	Source	Why it matters
Control-loop period	200 ms	ProjectConfig.h	Defines how often the controller can react to traffic changes.
Telemetry update period	1000 ms	ProjectConfig.h	Defines how fresh remote traffic data is when Node B decides.
Far ultrasonic threshold	100 cm	ProjectConfig.h	Detection window around the far sensor (placed ~40 m upstream).
Near ultrasonic threshold	100 cm	ProjectConfig.h	Detection window around the near sensor (at the stop line).
Minimum green time	5000 ms	ProjectConfig.h	Prevents unstable rapid switching.
Yellow transition time	2000 ms	ProjectConfig.h	Enforces a safety interval before the other side can become green.
Maximum green time	20000 ms	ProjectConfig.h	Prevents starvation of the waiting side.
Demand score formula	$3 * \text{queue} + 2 * \text{farOccupied} + 4 * \text{nearOccupied}$	TrafficTypes.h and traffic_logic.py	Explains how the controller compares both sides.
Example telemetry payload size	19 bytes	NodeMessaging.cpp	Shows that node-to-node communication remains compact.
Real telemetry TX evidence	<code>tx=RADIO_TX_OK</code>	Node A monitor	Confirms live telemetry transmission on hardware.
Real telemetry RX evidence	<code>source=LORA_RADIO</code>		
<code>stale=OFF</code>	Node B monitor	Confirms live telemetry reception on hardware.	

Table 6: Metrics table ready to place in the verification slide.

5. Slide-Ready Technical Numbers

Safe sentence for the presentation

Use this wording: The communication and timing values come from the implemented firmware configuration, while the current power numbers are preliminary engineering estimates for the prototype and will be replaced by instrument measurements in the final stage.

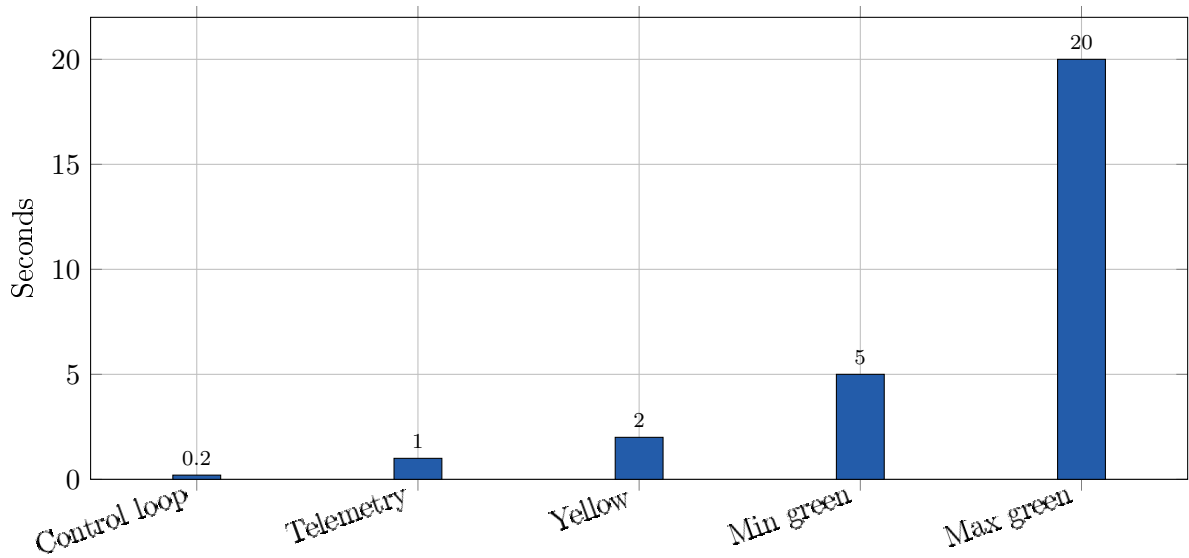


Figure 1: Current timing baseline used by the controller and telemetry path.

Technical quantity	Current value	Notes for the slide
ESP32 board target	Heltec WiFi LoRa 32 V3	Board used for both embedded nodes in the current prototype.
Microcontroller family	ESP32-S3	Current hardware platform used by the firmware build.
CPU clock	240 MHz	Board capability reported by the build environment.
RAM / Flash	320 KB RAM / 8 MB Flash	Board capability reported by the build environment.
Serial monitor speed	115200 bps	Current debug and bench-test serial speed.
Controller update interval	200 ms	Equivalent to a 5 Hz control loop.
Telemetry interval	1000 ms	Equivalent to a 1 Hz telemetry update rate.
Remote telemetry timeout	3000 ms	Old remote data is discarded after this timeout.
LoRa carrier frequency	868.0 MHz	Current configured regional radio setting.
LoRa bandwidth	125.0 kHz	Current radio configuration.
LoRa spreading factor	SF7	Current radio configuration.
LoRa coding rate	4/5	Current radio configuration.
LoRa output power	14 dBm	Current transmit-power configuration.
Example application payload	A,1,0,4,2,2,0,12345	Compact CSV-style telemetry packet.
Application payload size	19 bytes	Size of the example telemetry payload above.
Derived radio bit rate	5.47 kbps	Approximate useful bit rate from the current LoRa settings.
Estimated airtime for 19-byte payload	51 ms	Approximate radio airtime per packet at the current settings.

Table 7: Technical operating numbers that can be copied directly into a slide.

Power item	Current estimate	Power at 5 V	Status
Node A average current	$\tilde{120}$ mA	$\tilde{0.60}$ W	Engineering estimate, not yet instrument-measured.
Node A peak current during TX	$\tilde{160}$ mA	$\tilde{0.80}$ W	Engineering estimate for brief radio-transmit peaks.
Node B average current	$\tilde{140}$ mA	$\tilde{0.70}$ W	Engineering estimate with controller and light-driving logic active.
Node B peak current with radio and LEDs	$\tilde{180}$ mA	$\tilde{0.90}$ W	Engineering estimate for brief active peaks.
Two-node system average	$\tilde{260}$ mA	$\tilde{1.30}$ W	Preliminary whole-system estimate for the current prototype.

Table 8: Preliminary power-budget estimates. These values are placeholders for planning and must be replaced later by real instrument measurements.

6. Slide 4: Experiments And Results

Experiment	Setup	Observed result	Conclusion
Equal demand hold	Both sides have queue 2 from $t = 6$ s to $t = 11$ s.	Controller kept side A green.	Equal pressure does not trigger unnecessary switching.
Empty-lane yield	Side A becomes empty at $t = 6$ s while side B queue becomes 2.	Yellow at 6 s, side B green at 8 s.	Empty-lane detection yields at the first allowed instant.
Busier-side priority	Side A queue becomes 1 and side B queue becomes 4 after minimum green is already satisfied.	Yellow at 12 s in the scripted simulation baseline, and targeted test confirms 6 s to 8 s switching window after the trigger.	The busier side gets priority without skipping yellow.
Max-green enforcement	One side keeps demand for a long time while the other side continues waiting.	Yellow at 20 s, side B green at 22 s.	The maximum green cap prevents starvation.
Real LoRa A to B link	Node A transmits live telemetry and Node B listens over SX1262.	Node B changed from LORA_STALE to LORA_RADIO with stale=OFF and remoteQ=1.	Node-to-node communication was validated on real hardware.

Table 9: Experiment summary table for the result slide.

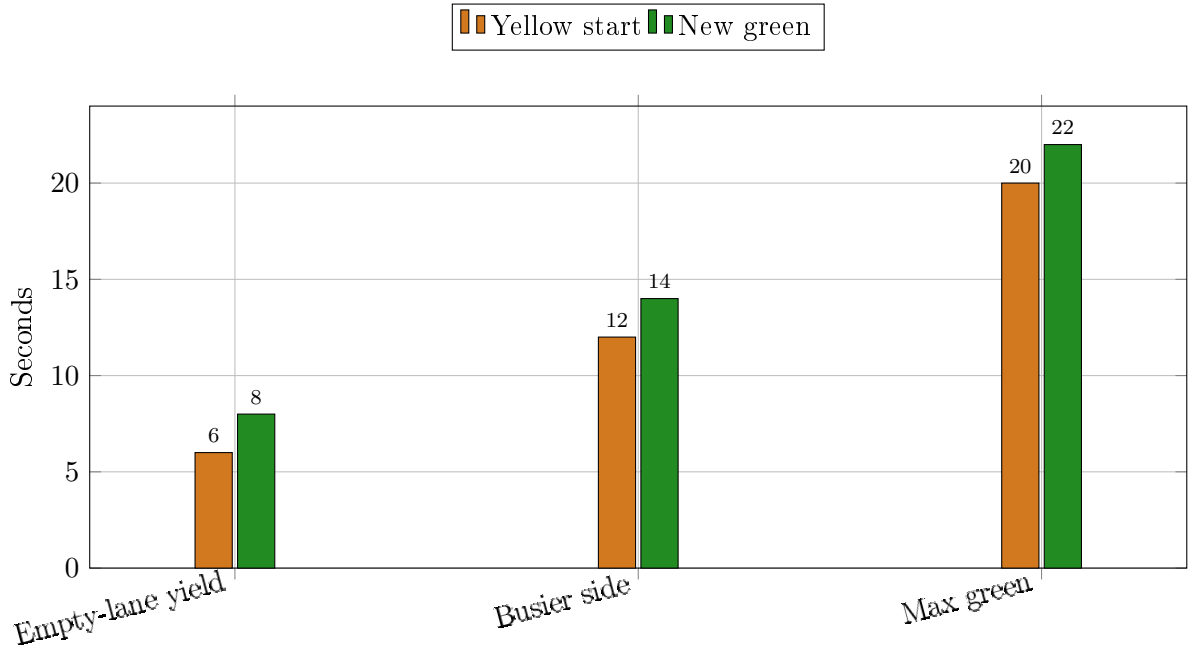


Figure 2: Observed switch timing in the main checkpoint experiments.

7. Slide 5: Hardware Evidence And Remaining Work

Area	Evidence	Interpretation	Status
Node A build and upload	PlatformIO build and upload succeeded on the real board.	The transmitter firmware is build-valid and flashable.	DONE
Node B build and upload	PlatformIO build and upload succeeded on the real board.	The receiver/controller firmware is build-valid and flashable.	DONE
Node A telemetry path	Node A produced queue-changing payloads and reported tx=RADIO_TX_OK.	Telemetry generation and radio transmission work on hardware.	DONE
LoRa communication	Node B reported source=LORA_RADIO, stale=OFF, and remoteQ=1 from live Node A traffic.	Real A-to-B packet reception was validated.	DONE
Ultrasonic sensing	Hardware integration started, but validation is not yet complete for all physical sensor paths.	Sensing remains an integration and calibration task.	IN PROGRESS
Emergency override on hardware	Logic exists in firmware, but the full real-hardware path is not yet fully validated.	Feature is implemented but not yet fully demonstrated physically.	PENDING TEST

Table 10: Current hardware-validation status to show honestly in the last slide.

Already done for the checkpoint	Remaining after the checkpoint
Adaptive controller validated in software	Full ultrasonic validation on both sides
Firmware build validated on both boards	Full LED and sensing integration together
Firmware upload validated on both boards	Emergency override validation on the real hardware path
Node A telemetry path validated on real hardware	End-to-end tests with physical sensing and actuation
Real LoRa communication validated between Node A and Node B	Threshold tuning and calibration

Table 11: Ready-made done-versus-remaining table for the closing slide.

8. Real Log Evidence

Node A Evidence

```
Node A ready.
[LoRa] backend: RadioLib SX1262
A STATUS | source=SERIAL_EMU | far=OCC | near=FREE | queue=1 | in=1 | out=0 | emergency=OFF
          | tx=RADIO_TX_OK | payload=A,1,0,1,0,1,0,62000
```

```
A STATUS | source=SERIAL_EMU | far=FREE | near=OCC | queue=1 | in=1 | out=0 | emergency=OFF
| tx=RADIO_TX_OK | payload=A,0,1,1,0,1,0,70000
A STATUS | source=SERIAL_EMU | far=FREE | near=FREE | queue=0 | in=1 | out=1 | emergency=
OFF | tx=RADIO_TX_OK | payload=A,0,0,1,1,0,0,79000
```

Node B Evidence

```
Node B ready.
[LoRa] backend: RadioLib SX1262
B STATUS | far=999.0cm/FREE | near=999.0cm/FREE | localQ=0 | remoteQ=0 | source=LORA_STALE
| stale=ON | green=A | phase=GREEN | emergency=OFF | priority=A | lights=A:GREEN B:RED
B STATUS | far=999.0cm/FREE | near=999.0cm/FREE | localQ=0 | remoteQ=1 | source=LORA_RADIO
| stale=OFF | green=A | phase=GREEN | emergency=OFF | priority=A | lights=A:GREEN B:RED
```

9. Sources Inside The Repository

- docs/checkpoint-demo.md: requirement list, metric table, and experiment structure
- docs/april-10-presentation-pack.md: slide-by-slide narrative
- docs/test-results-log.md: validated software and hardware results
- docs/april-10-readiness.md: current readiness state
- docs/project-stage-report.pdf: fuller written report

10. One-Sentence Closing

Closing sentence to say in the presentation: At this checkpoint, we validated the decision logic in software and the communication path on real hardware, so the remaining work is focused on physical sensing, actuation, and calibration rather than redesigning the system architecture.